21.04.2026

AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ DERSİ VİZE PROJE ÖDEVİ

ZEYNEP LÂL KAŞIKÇI - 22290070

github.com/lalkasikci

İÇİNDEKİLER

I. GİRİŞ 2

II. PROJE 1: VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA 2

A. Amaç 2

B. Yedekleme Stratejileri 2

C. Otomatik Yedekleme ve Zamanlama 2

D. Point-in-Time Restore 3

E. Database Mirroring 3

F. Felaketten Kurtarma Senaryoları 3

G. Yedeklerin Test Edilmesi 4

III. PROJE 2: VERİ TEMİZLEME VE ETL SÜREÇLERİ 4

A. Amaç 4

B. Veri Çıkarma 5

C. Veri Temizleme 5

D. Veri Dönüştürme 6

E. Veri Yükleme 6

IV. SONUÇ 7

1. GİRİŞ

Bu raporda, SQL tabanlı iki ayrı proje incelenmektedir. İlk proje, SQL dilinin temel komutlarını ve Veritabanı Yedekleme ve Felaketten Kurtarma Planına yönelik örneklerden oluşmaktadır. İkinci proje ise ham verilerin işlenmesi, temizlenmesi ve dönüştürülmesi sürecini ele alan bir veri işleme (ETL) yaklaşımını içermektedir.

---

2. PROJE 1: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

2.1 Amaç

Bu projenin amacı, SQL dilinin temel bileşenlerini kullanarak veritabanını yedeklemek ve felaketten kurtarmaktır. Bu kapsamda veri sorgulama, filtreleme, sıralama ve veritabanı yönetimi işlemleri ele alınmıştır.

2.2 Yedekleme Stratejilerinin Tasarlanması

Projede veri güvenliğini sağlamak amacıyla farklı yedekleme türleri birlikte kullanılmıştır. Bu sayede hem performans hem de veri kaybını minimize etme hedeflenmiştir.

Tam Yedekleme (Full Backup)

Tam yedekleme, veritabanının belirli bir andaki tüm durumunun eksiksiz şekilde yedeklenmesini sağlar. Projede haftalık periyotlarla tam yedekleme alınacak şekilde bir strateji belirlenmiştir. Bu yedekler, sistemin tamamen geri yüklenmesi gereken durumlar için temel referans noktası olarak kullanılmıştır.

Fark Yedekleme (Differential Backup)

Fark yedekleme, son alınan tam yedekten sonra değişen verileri içerir. Projede günlük olarak fark yedekleri alınarak veri kaybı riski azaltılmış ve geri yükleme süresi optimize edilmiştir.

2.3 Otomatik Yedekleme ve Zamanlama

Projede yedekleme işlemlerinin manuel olarak yapılması yerine otomatik hale getirilmesi hedeflenmiştir. Tam yedekleme işlemleri haftalık olarak planlanmıştır

- Fark yedeklemeler günlük olarak gerçekleştirilmiştir
- Transaction log yedekleri daha kısa aralıklarla (örneğin saatlik) alınmıştır

Bu yapı sayesinde insan hatası minimize edilmiş ve düzenli yedekleme sağlanmıştır.

2.4 Point-in-Time Restore (Belirli Zamana Geri Dönüş)

Proje kapsamında en kritik konulardan biri, veritabanının belirli bir zamandaki durumuna geri döndürülebilmesidir. Bu amaçla transaction log yedekleri kullanılarak point-in-time restore işlemleri gerçekleştirilmiştir.

Bu senaryoda:

- Veritabanı önce en son tam yedekten geri yüklenmiştir
- Ardından ilgili fark yedeği uygulanmıştır
- Son olarak transaction log yedekleri kullanılarak belirli bir zamana kadar geri yükleme yapılmıştır

Bu yöntem sayesinde veri kaybı minimum seviyeye indirilmiştir.

2.5 Database Mirroring

Sistem sürekliliğini sağlamak amacıyla veritabanı yedeklemenin yanı sıra anlık kopyalama (mirroring) yaklaşımı da ele alınmıştır. Database Mirroring ile veritabanının bir kopyası farklı bir sunucuda tutulmuş ve ana sistemde oluşabilecek bir hata durumunda yedek sistemin devreye girmesi sağlanmıştır.

SQLQuery12.sql

C: > Users > user > AppData > Local > Temp > 8fba6441-fda7-43dc-bde2-8b349c871493_proje1.zip.493 > proje1 > proje1 > SQLQuery12.sql

```sql
RESTORE DATABASE BackupDemoDB
FROM DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER01\MSSQL\Backup\BackupDemoDB.bak'
WITH REPLACE, RECOVERY;
GO
```

Bu yapı:

- Yüksek erişilebilirlik sağlar
- Sistem kesinti süresini azaltır
- Kritik sistemler için güvenli bir alternatif sunar

2.5 Felaketten Kurtarma Senaryoları

Projede olası veri kaybı durumları için çeşitli senaryolar oluşturulmuş ve bu senaryolar üzerinden kurtarma işlemleri test edilmiştir.

Senaryo 1: Veri Silinmesi

Kullanıcı hatası ile veri silinmesi durumunda:

- Transaction log yedekleri kullanılarak veri geri getirilmiştir

Senaryo 2: Veritabanı Bozulması

Veritabanının zarar görmesi durumunda:

- Tam yedek + fark yedek kullanılarak sistem yeniden oluşturulmuştur

Senaryo 3: Sistem Çökmesi

Sunucu seviyesinde bir hata durumunda:

- Yedek sunucu (mirroring) devreye alınmıştır.

SQLQuery8.sql

C: > Users > user > AppData > Local > Temp > c106f99c-719b-4c0d-9e30-a273a5108e97_proje1.zip.e97 > proje1 > proje1 > SQLQuery8.sql

```sql
BACKUP DATABASE BackupDemoDB
TO DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER01\MSSQL\Backup\BackupDemoDB.bak'
WITH INIT, STATS = 10;
GO
```

2.6 Yedeklerin Test Edilmesi

Yedekleme işleminin güvenilir olması kadar, yedeklerin çalışır durumda olması da kritik öneme sahiptir. Bu nedenle projede alınan yedekler düzenli olarak test edilmiştir.

Bu kapsamda:

- Yedekler farklı bir ortamda restore edilmiştir
- Veri bütünlüğü kontrol edilmiştir
- Hatalı veya eksik yedekler tespit edilmiştir

Bu testler sayesinde olası bir felaket durumunda yedeklerin sorunsuz çalışacağı garanti altına alınmıştır.

---

3. PROJE 2: Veri Temizleme ve ETL Süreçleri Tasarımı

3.1 Amaç

Çalışmanın temel amacı, ham verilerin doğrudan kullanılmasından kaynaklanabilecek hataları önlemek, veri kalitesini artırmak ve karar destek süreçlerinde kullanılabilecek güvenilir veri setleri oluşturmaktır.

3.2 Veri Çıkarma

ETL sürecinin ilk aşamasında, farklı veri kaynaklarından elde edilen veriler herhangi bir değişikliğe uğratılmadan sisteme alınmıştır. Bu veriler ham veri olarak adlandırılmakta ve genellikle eksik, hatalı veya tutarsız bilgiler içerebilmektedir.

Bu aşamada veriler, analiz edilmeden önce saklanmak üzere “raw” katmanına aktarılmıştır. Bu yaklaşım sayesinde:

- Orijinal veri korunmuş
- Geriye dönük kontrol imkânı sağlanmış
- Veri kaybı riski azaltılmıştır

SQLQuery19.sql

C: > Users > user > Downloads > proje2 > proje2 > SQLQuery19.sql

```sql
CREATE TABLE DataQualityLog (
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    RawID INT,
    ColumnName NVARCHAR(100),
    ErrorType NVARCHAR(100),
    ErrorDetail NVARCHAR(500),
    LogDate DATETIME DEFAULT GETDATE()
);
```

3.4 Veri Temizleme

Ham veriler doğrudan kullanılabilir durumda olmadığından, bu aşamada veri temizleme işlemleri gerçekleştirilmiştir. SQL sorguları kullanılarak veri içerisindeki hatalı ve eksik kayıtlar tespit edilmiştir.

Bu kapsamda yapılan işlemler:

- **Eksik veri kontrolü:** Zorunlu alanların boş olup olmadığı kontrol edilmiştir (örneğin FullName)

```sql
INSERT INTO DataQualityLog (RawID, ColumnName, ErrorType, ErrorDetail)
SELECT RawID, 'FullName', 'Missing Data', 'FullName boş veya eksik'
FROM StgCustomers
WHERE ErrorMessage LIKE '%FullName boş veya eksik%';
```

- **Format kontrolü:** E-posta, telefon ve tarih gibi alanların doğru formatta olup olmadığı doğrulanmıştır

```sql
UPDATE StgCustomers
SET
    IsValid = 0,
    ErrorMessage = COALESCE(ErrorMessage + '; ', '') + 'Geçersiz email formatı'
WHERE Email IS NULL
   OR Email NOT LIKE '%_@_%._%';
```

- **Tutarsız veri tespiti:** Aynı verinin farklı biçimlerde girilmesi kontrol edilmiştir
- **Mükerrer kayıt kontrolü:** Tekrarlayan (duplicate) kayıtlar belirlenmiştir

Ayrıca tespit edilen hatalar doğrudan silinmek yerine **DataQualityLog** tablosuna kaydedilmiştir. Bu sayede:

- Hataların kaydı tutulmuş
- Veri kalitesi izlenebilir hale getirilmiş
- Sistem daha şeffaf hale getirilmiştir

3.5 Veri Dönüştürme

Temizleme işlemlerinden sonra veriler, hedef sistemin gereksinimlerine uygun hale getirilmiştir. Bu aşamada farklı kaynaklardan gelen veriler standart bir yapıya dönüştürülmüştür.

Gerçekleştirilen başlıca işlemler:

- String formatındaki tarihlerin DATE tipine dönüştürülmesi
- Telefon numaralarının standart formata getirilmesi
- Büyük/küçük harf uyumsuzluklarının giderilmesi
- Veri tiplerinin hedef tablo yapısına uygun hale getirilmesi

Bu işlemler sonucunda veriler hem teknik hem de anlamsal olarak tutarlı hale getirilmiştir.

3.6 Veri Yükleme

Dönüştürülen ve temizlenen veriler, ETL sürecinin son aşamasında hedef veritabanına aktarılmıştır. Bu aşamada yalnızca doğrulanmış ve hatasız verilerin yüklenmesine dikkat edilmiştir.

Bu yaklaşım sayesinde:

- Hatalı verilerin sisteme girişi engellenmiş
- Veri bütünlüğü korunmuş
- Analiz için güvenilir veri setleri oluşturulmuştur

```sql
UPDATE d
SET
    d.FullName = s.FullName,
    d.FirstName = s.FirstName,
    d.LastName = s.LastName,
    d.Phone = s.Phone,
    d.City = s.City,
    d.BirthDate = s.BirthDate,
    d.SourceSystem = s.SourceSystem,
    d.UpdateDate = GETDATE()
FROM DimCustomer d
INNER JOIN StgCustomers s
    ON d.Email = s.Email
WHERE s.IsValid = 1;
```

4. Sonuç

Bu çalışma kapsamında iki farklı proje gerçekleştirilmiş olup, her biri veritabanı yönetimi ve veri işleme süreçlerinin farklı yönlerini ele almaktadır.

İlk proje, SQL dilinin temel kullanımına odaklanmaktadır. Bu kapsamda veri sorgulama, filtreleme, sıralama ve veritabanı yönetimi işlemleri uygulanmıştır. Ayrıca veritabanı yedekleme ve geri yükleme işlemleri ile sistem güvenliğine yönelik temel yaklaşımlar ele alınmıştır. Bu proje, SQL komutlarının öğrenilmesi ve veritabanı üzerinde temel işlemlerin uygulanması açısından bir altyapı oluşturmaktadır.

İkinci proje ise veri temizleme ve ETL süreçlerine odaklanmaktadır. Bu kapsamda ham verilerin sisteme alınması, hatalı ve eksik verilerin tespit edilmesi, veri dönüştürme işlemlerinin gerçekleştirilmesi ve temiz verilerin hedef tablolara yüklenmesi sağlanmıştır. Ayrıca veri kalitesi kontrolü yapılmış ve hatalı veriler loglanarak analiz edilebilir hale getirilmiştir. Bu proje, veri işleme süreçlerinin daha sistematik ve kontrollü bir şekilde yürütülmesini sağlamaktadır.